- Your application is built with WCF.
- Your application is large, with many advanced MVC features.

If you have an ASP.NET Web Forms application, attempting to convert it directly to ASP.NET Core isn't advisable. Web Forms is inextricably tied to System.Web.dll and IIS, so it will likely never be available in ASP.NET Core. Converting an application to ASP.NET Core effectively involves rewriting the application from scratch, not only shifting frameworks, but also potentially shifting design paradigms.

All is not lost, however. Blazor server provides a stateful, component-based application that's *similar* to the Web Forms application model. You may be able to gradually migrate your Web Forms application page by page to an ASP.NET Core Blazor server application.[4] Alternatively, you could slowly introduce web API concepts into your Web Forms application, reducing the reliance on legacy Web Forms constructs such as ViewState, with the goal of ultimately moving to an ASP.NET Core web API application.

Windows Communication Foundation (WCF) is only partially supported in ASP.NET Core. It's possible to build client-side WCF services using the libraries provided by ASP.NET Core (https://github.com/dotnet/wcf) and to build server-side WCF services by using the Microsoft-supported community-driven project CoreWCF.[5] These libraries don't support all the APIs available in .NET Framework WCF (distributed transactions and some message security formats, for example), so if you absolutely need those APIs, it may be best to avoid ASP.NET Core for now.

TIP If you like WCF's contract-based RPC-style of programming but don't have a hard requirement for WCF itself, consider using gRPC instead. gRPC is a modern RPC framework with many concepts that are similar to WCF, and it's supported by ASP.NET Core out of the box (http://mng.bz/wv9Q).